Name: Rajvardhan Patil Roll no:2401081 Batch :T21

EXPERIMENT NO. 07

Aim: To implement monitoring using Prometheus / Grafana / Nagios

LO Mapping: LO5

LO5: Manage the ML lifecycle end-to-end and build automated workflows using MLflow and Airflow, backed by monitoring tools such as Prometheus, Grafana, or Nagios.

Theory

Continuous monitoring is the ongoing practice of detecting, reporting on, and responding to problems or attacks within an organization's IT infrastructure. Its importance becomes clear once an application goes live on a server — from that point on, monitoring is what keeps watch over the infrastructure and makes sure issues are dealt with quickly and appropriately.

Why Use Nagios?

Nagios is worth adopting for several practical reasons:

- Identifies a wide range of network and server issues as they arise
- Helps trace problems back to their root cause, enabling a permanent fix rather than a temporary workaround
- Actively watches over the entire infrastructure as well as key business processes
- Makes it easier to monitor and troubleshoot server performance
- Supports proactive planning for infrastructure upgrades before ageing systems start to fail
- Helps preserve the security and availability of services
- Can trigger automatic fixes for certain problems during critical situations

Nagios itself is one of the most widely used open-source monitoring platforms, capable of watching applications, services, and entire IT environments. It helps teams spot and resolve infrastructure problems before those problems start affecting core business operations.

To reach out to remote machines, Nagios relies on an agent. On Linux hosts this agent is NRPE (Nagios Remote Plugin Executor), which runs plugins on the remote system on Nagios's behalf. The Windows counterpart is NSClient++, a Windows service that listens for requests from the Nagios server and carries out local checks — disk space, CPU load, memory usage, running services, and so on.

In practice, NSClient++ sits on the monitored Windows machine and waits for the Nagios server to contact it, typically over the NRPE protocol (which NSClient++ also understands). Whenever Nagios needs to check something on that host, it sends a request; NSClient++ runs the check locally and reports the result back to Nagios.

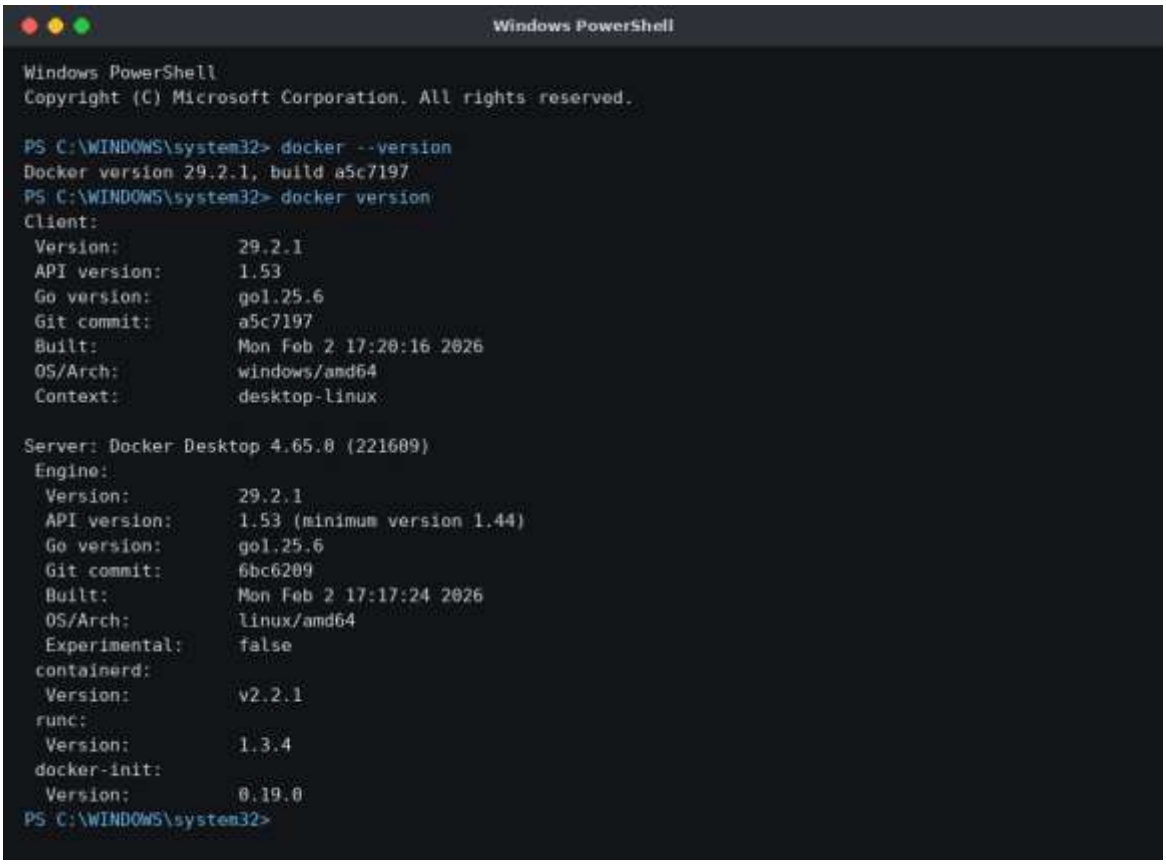
A Note on Running Nagios on Windows

Nagios Core is designed for Linux and cannot be installed natively on a Windows machine. The practical workaround on Windows is to run the Nagios Core server inside a Docker container — using Docker Desktop with the WSL2 backend — while monitoring the Windows hosts themselves through NSClient++, which plays the same role on Windows that NRPE plays on Linux. All the commands that follow are run from Windows PowerShell or Command Prompt.

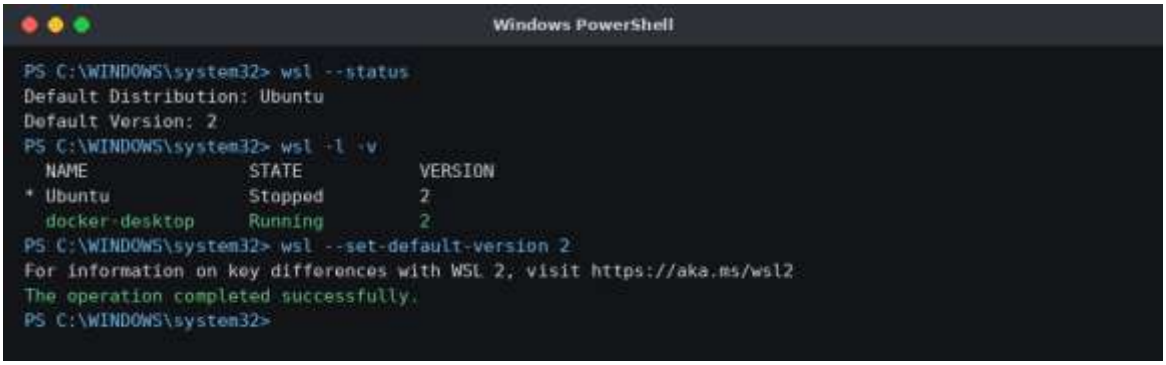
Name: Rajvardhan Patil Roll no:2401081 Batch :T21

Step 1 — Install Docker Desktop

Download and install Docker Desktop for Windows, making sure the WSL2-based engine is enabled. Confirm the installation is working from PowerShell:



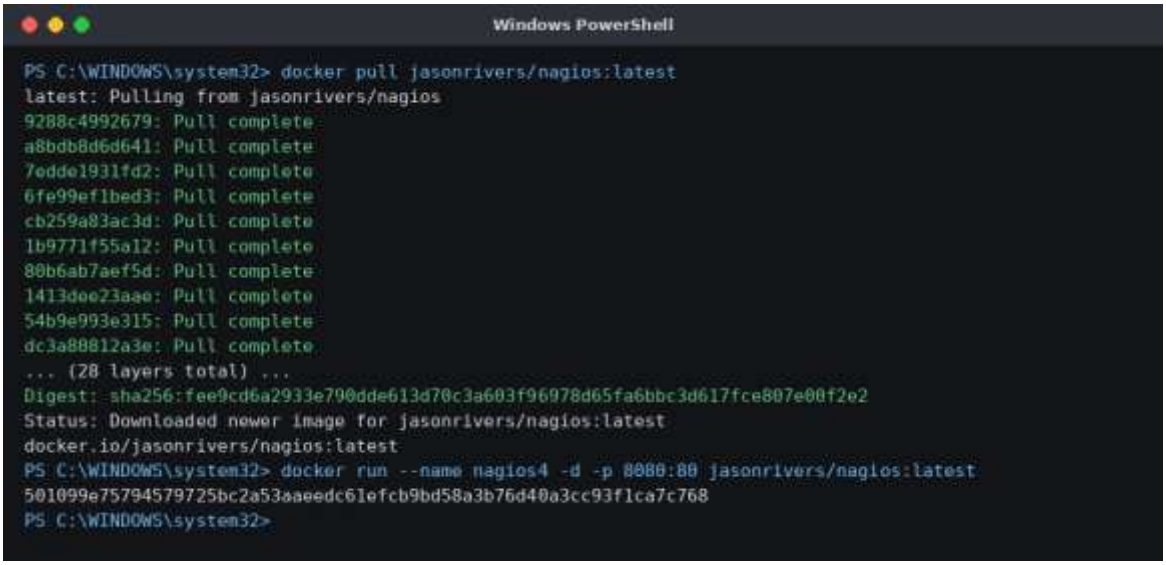
Also confirm that the WSL2 feature itself is enabled and active:



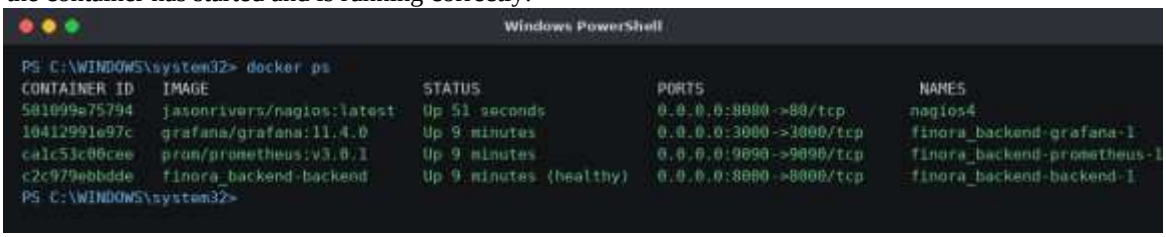
Step 2 — Pull and Run the Nagios Core Docker Image

Pull a ready-made Nagios Core image from Docker Hub and start it as a container, mapping its web port through to the host machine:

Name: Rajvardhan Patil Roll no:2401081 Batch :T21

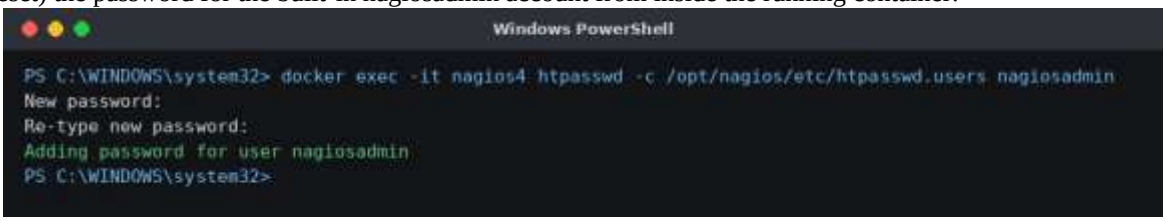


Confirm the container has started and is running correctly:



Step 3 — Set the Nagios Admin Password

Set (or reset) the password for the built-in nagiosadmin account from inside the running container:



Enter and confirm a password when prompted (for example, 123456).

Step 4 — Access the Nagios Web Interface

Open the Nagios web interface from a browser on the Windows host:

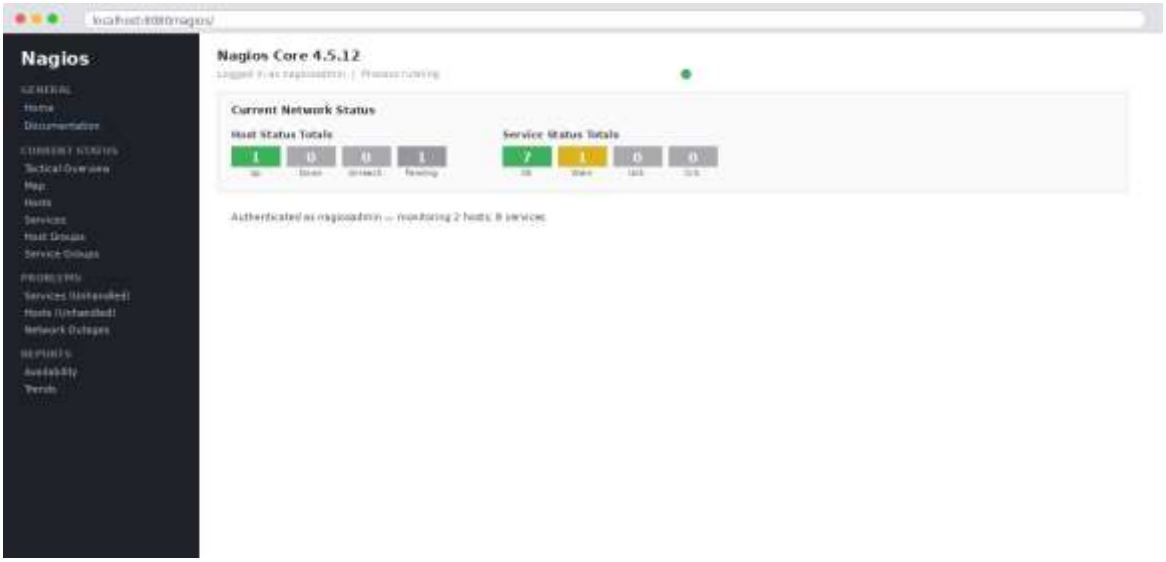
```
http://localhost:8080/nagios/
```

When prompted for authentication, sign in with:

- Username: nagiosadmin
- Password: 123456 (the one set during configuration)

Once logged in, the dashboard summarises the current health of all monitored hosts and services:

Name: Rajvardhan Patil Roll no:2401081 Batch :T21



Step 5 — Install NSClient++ on the Windows Remote Host

On the Windows machine that needs to be monitored, download and silently install NSClient++ — the Windows equivalent of the NRPE agent:

```
Invoke-WebRequest -Uri "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi" -OutFile "NSCP.msi" Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL MODULES=CheckExternalScripts,NRPEServer /qn' -Wait
```

Confirm the service installed and is running:



Step 6 — Configure NSClient++

Open the NSClient++ configuration file and add the Nagios server's IP address to the list of allowed hosts, so the server is permitted to query this machine:

Name: Rajvardhan Patil Roll no:2401081 Batch :T21

```
File  Edit  View

# If you want to fill this file with all available options run the following command:
#   nscp settings --generate --add-defaults --load-all
# If you want to activate a module and bring in all its options use:
#   nscp settings --activate-module <MODULE NAME> --add-defaults
# For details run: nscp settings --help

; in flight - TODO
[/modules]

; Load on startup - If the module should be loaded on startup
NRPEServer = enabled

; Load on startup - If the module should be loaded on startup
WEBServer = enabled

; Load on startup - If the module should be loaded on startup
CheckDisk = enabled

; Load on startup - If the module should be loaded on startup
CheckSystem = enabled

; Load on startup - If the module should be loaded on startup
CheckEventLog = enabled

; Load on startup - If the module should be loaded on startup
CheckHelpers = enabled

; Load on startup - If the module should be loaded on startup
CheckExternalScripts = enabled

; Load on startup - If the module should be loaded on startup
CheckNSCP = enabled

; in flight - TODO
[/settings/NRPE/server]

verify mode = none

tls version = tlsv1.2+

insecure = true

allowed ciphers = ADH

; in flight - TODO
[/settings/default]

allowed hosts = 127.0.0.1,172.17.0.2,192.168.1.100

Ln 40, Col 1  1203 characters  Plain text
```

Here 192.168.1.100 represents the IP address of the Nagios server (the Docker host). Once the file is saved, restart the NSClient++ service for the change to take effect:

```
Administrator: Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> Restart-Service nscp
PS C:\WINDOWS\system32> Get-Service nscp

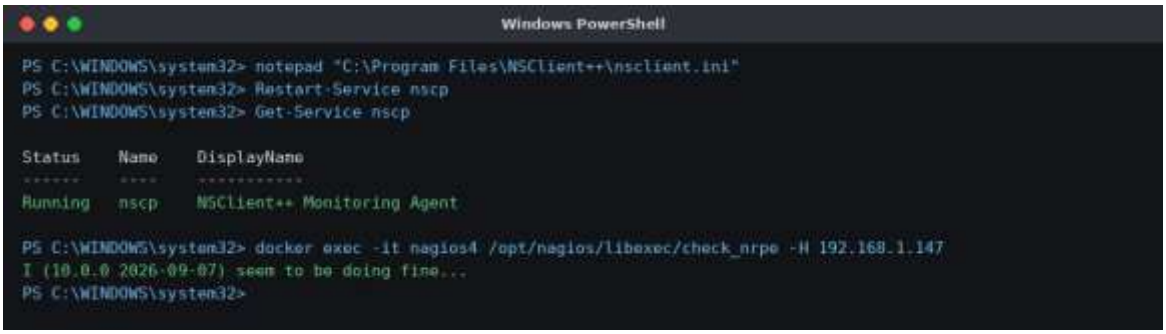
Status   Name    DisplayName
------   ----    -----------
Running  nscp    NSClient++ Monitoring Agent

PS C:\WINDOWS\system32>
```

Name: Rajvardhan Patil Roll no:2401081 Batch :T21

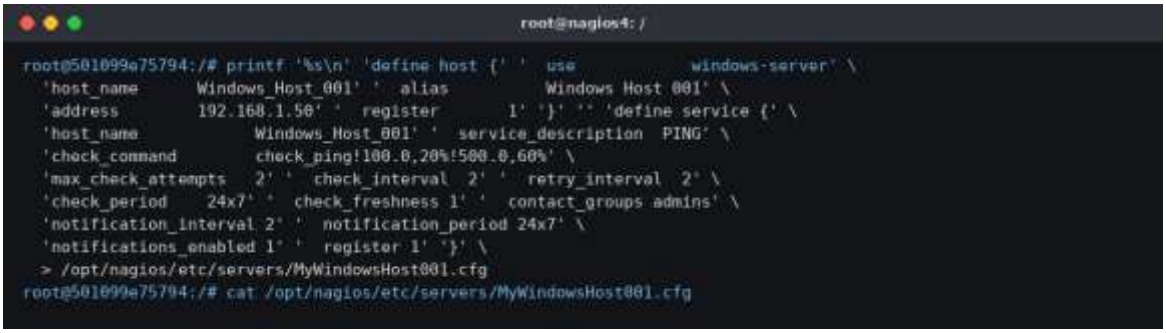
Step 7 — Verify Connectivity from Nagios

From the Nagios server (running inside the Docker container), issue an NRPE check against the Windows remote host to confirm the two machines can talk to each other. In this example, 192.168.1.147 is the remote Windows host's IP address:



Step 8 — Add the Windows Host in Nagios

Create a configuration file for the new host inside the Nagios container, along with a service definition that continuously checks whether the host responds to a PING:




```
define host {
  use             windows-server
host_name       Windows_Host_001
alias           Windows Host 001
address         192.168.1.50
  register        1
}

define service {
  host_name               Windows_Host_001
  service_description     PING
  check_command           check_ping!100.0,20%!500.0,60%
  max_check_attempts      2
check_interval          2
retry_interval          2
check_period            24x7
check_freshness         1
contact_groups          admins
notification_interval   2
notification_period     24x7
notifications_enabled   1
register                1
}
```

Validate the configuration file for errors, then restart the Nagios container so the changes take effect:

Name: Rajvardhan Patil Roll no:2401081 Batch :T21

```
root@nagios4: /
root@501099e75794:/# /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
License: GPL

Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
       Checked 8 services.
Warning: Host 'Windows_Host_001' has no default contacts or contactgroups defined!
       Checked 2 hosts.
       Checked 1 host groups.
       Checked 1 contacts.
       Checked 23 commands.
       Checked 5 time periods.
Checking for circular paths...
       Checked 2 hosts.
Checking global event handlers...
Checking misc settings...

Total Warnings: 1
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check
root@501099e75794:/#
```

Step 9 — Check the Host in the Nagios Web Interface

Open the Nagios web interface again (http://localhost:8080/nagios/) and confirm that the new Windows host has been added and is being actively monitored, under Current Status → Services and Current Status → Hosts:



Name: Rajvardhan Patil Roll no:2401081 Batch :T21



Conclusion

This experiment set up an end-to-end monitoring pipeline using Nagios: the Nagios Core server was containerised with Docker on a Windows host, a remote Windows machine was made monitorable using NSClient++, and a custom host/service definition was added so its availability could be tracked from the Nagios web dashboard. This demonstrates the core monitoring workflow used to keep infrastructure — and, by extension, deployed ML systems — healthy and observable in production.